

Relatório — Camada de Banco de Dados

Projeto Pokédex (trabalho em grupo)

Aluno: Luís Leão

Componente individual: Banco de Dados (SQLite + ORMLite)

Data: junho de 2026

1. Objetivo

Este relatório descreve a camada de banco de dados do projeto Pokédex, de minha responsabilidade. A partir de um arquivo CSV com os 151 Pokémon da primeira geração, construí a camada de acesso a dados do sistema: a conexão com o banco, os repositórios que realizam as operações de persistência e a rotina de carga que popula o banco. Foi utilizado o ORMLite (mapeamento objeto-relacional) sobre SQLite, seguindo o padrão Repository apresentado no tutorial da disciplina. As classes de modelo (entidades) pertencem à etapa de modelagem do projeto; os repositórios desta camada operam sobre elas.

2. Tecnologias utilizadas

- SQLite — banco de dados em arquivo único, sem necessidade de servidor.
- ORMLite 6.1 — mapeamento objeto-relacional (ORM); a persistência é feita por métodos (create, query) em vez de SQL escrito à mão.
- Driver sqlite-jdbc 3.46.0.0 — conexão JDBC com o SQLite, usada por baixo pelo ORMLite.
- SLF4J (api e simple) 2.0.13 — biblioteca de log exigida pelo ORMLite.
- BlueJ — ambiente de desenvolvimento, com as bibliotecas adicionadas via Preferências › Bibliotecas.

3. Arquitetura adotada

A camada que desenvolvi é composta por três responsabilidades, seguindo o padrão do tutorial:

- Conexão (classe Database): gerencia a abertura e o fechamento da conexão com o arquivo SQLite.
- Repositórios (padrão Repository): uma classe por entidade, com as operações sobre o banco — create, loadFromId, loadAll e consultas. As demais partes do sistema acessam os dados apenas por estes repositórios, sem escrever SQL.
- Carga de dados (classe PopularBanco): lê os arquivos CSV e popula o banco através dos repositórios.

Os repositórios operam sobre as classes de modelo (Pokemon, Golpe, Nature e TipoEfetividade), que fazem parte da etapa de modelagem do projeto e carregam as anotações de mapeamento do ORMLite.

4. Arquivos desta camada

Arquivo	Função
Database.java	Gerencia a conexão com o arquivo SQLite (abrir/fechar).

Arquivo	Função
PokemonRepository.java	Operações sobre os Pokémon: create, loadFromId, loadAll, busca por nome, filtro por tipo e cálculo de fraquezas/vantagens.
GolpeRepository.java	Operações sobre o catálogo de golpes.
NatureRepository.java	Operações sobre as naturezas.
TipoEfetividadeRepository.java	Operações sobre a tabela de efetividade entre tipos (TypeChart).
PopularBanco.java	Rotina de carga: lê os CSVs e popula o banco via repositórios; gera o arquivo pokedex.db.
pokemons.csv	Dados-base: os 151 Pokémon (número, nome, tipos, atributos e imagem).
golpes.csv	Catálogo de golpes (nome, tipo, poder, precisão, categoria).
pokemon_golpes.csv	Associação entre cada Pokémon e o seu golpe característico.
LEIA-ME.md	Instruções de instalação das bibliotecas e de execução.

5. Origem e tratamento dos dados

A base foi um arquivo CSV com os 151 Pokémon da primeira geração (número, nome, dois tipos, seis atributos e a URL da imagem), obtido de um repositório público no GitHub. Sobre essa base foram feitos os seguintes tratamentos e complementos:

- Padronização do cabeçalho do CSV (colunas sem espaços nem acentos) e leitura em UTF-8, preservando caracteres especiais (por exemplo, Nidoran♀ e Nidoran♂).
- Tipo secundário ausente passa a ser nulo no banco, em vez de texto vazio.
- Associação de um golpe característico a cada Pokémon (uso visual, exibido no card).
- Inclusão de dados canônicos que não constavam no CSV: a tabela de efetividade entre os 18 tipos (apenas os confrontos não neutros) e as 25 naturezas.

6. Processo de desenvolvimento

1. Definição do escopo e da fonte de dados (o CSV dos 151 Pokémon).
2. Tratamento do CSV (cabeçalho padronizado e codificação UTF-8).
3. Escolha do padrão de persistência (ORMLite + Repository, conforme o tutorial da disciplina).
4. Implementação da classe de conexão (Database).
5. Implementação dos repositórios (operações CRUD e consultas) sobre as classes de modelo.
6. Complemento dos dados canônicos (golpe por Pokémon, tabela de tipos e naturezas).
7. Implementação da rotina de carga (PopularBanco), que lê os CSVs e persiste tudo via ORMLite.
8. Validação dos dados gerados (contagens e verificação de integridade).

7. Resultado

Ao executar a rotina de carga, é gerado o arquivo `pokedex.db` já populado. Os volumes obtidos:

Conteúdo	Quantidade
Pokémon cadastrados	151
Golpes no catálogo	91
Natures	25
Confrontos na tabela de efetividade	120

A verificação confirmou que todos os Pokémon foram importados, que cada um possui exatamente um golpe associado e que não há violações de integridade referencial nos dados.

8. Integração com o restante do projeto

Esta camada é a fonte de dados que alimenta as demais partes. A interface não acessa o banco diretamente nem escreve SQL: ela cria um objeto `Database` e os repositórios e, a partir deles, obtém objetos prontos para exibição — por exemplo, `loadAll` para a grade, `loadByName` para a busca, `loadByType` para o filtro e o cálculo de fraquezas para o painel de análise.

9. Como executar

9. Adicionar ao BlueJ as cinco bibliotecas (`sqlite-jdbc`, `ormlite-core`, `ormlite-jdbc`, `slf4j-api` e `slf4j-simple`).
10. Manter os três arquivos CSV na pasta do projeto.
11. Compilar as classes.
12. Executar o método `main` da classe `PopularBanco`, que cria e popula o arquivo `pokedex.db`.

10. Considerações finais

A camada de banco de dados está concluída e validada, seguindo o padrão ORM e `Repository` solicitado. Eventuais extensões (no mesmo padrão) dependem de decisões da equipe, como permitir a edição de golpe ou `nature` na montagem do time e, caso necessário, a persistência do time montado pelo usuário.